Softwareprojekt

CampusKompass

Vereinfachte Code- und Anforderungspräsentation

Agenda

1. Ziel und Funktionen
2. User Stories und Use Cases
3. MoSCoW-Analyse
4. Architektur
5. Bedienung und Weboberfläche

1. Ziel und Funktionen

Ziel

CampusKompass bündelt die wichtigsten Informationen für einen Studientag an den Standorten Schmalkalden und Gera in einer lokalen Weboberfläche. Die Anwendung reduziert den Wechsel zwischen mehreren externen Informationsquellen und ordnet die Daten dem ausgewählten Datum zu.

Funktionen

- Studienplan nach Datum filtern und Termine mit Zeit, Veranstaltung, Lehrkraft, Ort und Raum anzeigen
- Mensaplan des passenden Campus über den lokalen Server laden
- Wetter für Campus, Datum und Uhrzeit anzeigen
- Campusabhängiges Ziel für Route und ÖPNV-Fahrplanauskunft setzen

CampusKompass auf einen Blick

- Lokales Webdashboard für Studierende
- Studienplan, Mensaplan, Wetter und Campusroute in einer Oberfläche
- Frontend mit HTML, CSS und JavaScript
- Python-Server für statische Dateien und den Mensa-Proxy

2. User Stories und Use Cases

User Stories

- **CAMPUS-01:** Studienplan für ein Datum anzeigen
- **CAMPUS-02:** Passenden Mensaplan anzeigen
- **CAMPUS-03:** Wetter am Campus anzeigen
- **CAMPUS-04:** Route zum Campus planen
- **CAMPUS-05:** Fahrplan für Datum und Uhrzeit öffnen
- **CAMPUS-06:** Auswahl auf die aktuelle Zeit setzen

Use Cases: CampusKompass

ID	Use Case	Hauptakteur
UC-01	Dashboard öffnen	Student/in

3. MoSCoW-Analyse

Die MoSCoW-Methode priorisiert Anforderungen nach ihrer Bedeutung für das MVP:

- **Must have:** unverzichtbar für die erste Version
- **Should have:** wichtig, aber verschiebbar
- **Could have:** nützliche Erweiterung
- **Won't have:** in dieser Version bewusst ausgeschlossen

Priorität	Beispiele
Must have	Studienplan für ein Datum anzeigen
Should have	Mensaplan und Wetter am passenden Campus
Could have	Campusroute und Fahrplanauskunft
Won't have	Nicht priorisierte Erweiterungen in dieser Version

4. Architektur

Systemarchitektur

```
Browser
  |
  +--> index.html / JavaScript
  |       +--> study-plan.json
  |       +--> Open-Meteo
  |       +--> Google Maps
  |       +--> /api/mensa
  |
  +--> server.py
          +--> statische Dateien
          +--> Proxy zum Studierendenwerk Thüringen
```

CampusKompass: Ablauf

1. Nutzer wählt ein Datum.
2. Der Studienplan wird nach diesem Datum gefiltert.
3. Der Studienort bestimmt Mensa, Wetterstandort und Routenziel.
4. Mensaplan und Wetter werden geladen.
5. Die Campusroute wird als Wegbeschreibung angezeigt.

Der Proxy in `server.py` verhindert, dass der Browser den Mensa-Endpunkt direkt wegen CORS ansprechen muss.

4.1 Modellierung: Klassendiagramm Backend

Das Backend besteht aus dem Modul `server.py` und dem spezialisierten Handler `DashboardHandler`:

```
server.py
  +-- main()
  +-- ThreadingHTTPServer
          +-- DashboardHandler
                  +-- do_GET()
                  +-- /api/mensa -> Studierendenwerk Thüringen
```

`DashboardHandler` erweitert `SimpleHTTPRequestHandler`. Dadurch werden statische Dateien ausgeliefert; der zusätzliche API-Pfad validiert Datum und Mensa-ID und leitet die Anfrage als POST weiter.

Quelle: `modellierung/klassendiagramm_backend.puml`

4.2 Modellierung: Klassendiagramm Frontend

Das JavaScript in `index.html` ist prozedural organisiert. Für die Modellierung wird es als `CampusDashboard` mit seinen Zuständigkeiten dargestellt:

```
CampusDashboard
  +-- StudySession[] aus study-plan.json
  +-- loadStudyPlan() / renderStudyPlan()
  +-- loadMenu() -> Meal[]
  +-- loadWeather() -> WeatherData
  +-- showRoute() -> RouteDestination
```

Die Funktionen wählen anhand des Datums Mensa, Wetterstandort und Routenziel aus und aktualisieren anschließend die Oberfläche.

Quelle: `modellierung/klassendiagramm_frontend.puml`

4.3 Modellierung: Paketdiagramm

```
CampusKompass
  +-- Frontend: index.html
  +-- Backend: server.py
  +-- Daten: study-plan.json, assets/

Frontend -> Backend -> Studierendenwerk Thüringen
Frontend -> Open-Meteo
Frontend -> Google Maps
Frontend -> VMT Fahrplanauskunft
```

Die Paketgrenzen machen sichtbar, dass nur der Mensa-Aufruf über das Backend läuft. Wetter, Karten, Fahrplan und Studienplandaten werden vom Frontend verarbeitet.

Quelle: `modellierung/paketdiagramm.puml`

5. Bedienung und Weboberfläche

Bedienablauf

1. `python .\server.py` starten und `http://127.0.0.1:8000/` öffnen.
2. Datum und Abfahrtszeit im Bereich „Tagesplanung“ auswählen.
3. Studienplan, Mensa und Wetter für die Auswahl prüfen.
4. Im Fahrplan-Widget Startort und das angezeigte Campusziel verwenden.
5. Für eine Autofahrt Startpunkt eingeben oder den Standort freigeben.
6. Mit „Jetzt“ jederzeit zur aktuellen Zeit zurückkehren.

Oberfläche

- Kopfbereich mit Hochschullinks und aktuellem Datum/Uhrzeit
- Tagesplanung als zentrale Eingabe für Datum und Zeit
- Karten für Wetter, Mensa, Studienplan, Fahrplan und Route

Tests und Erweiterbarkeit

Durchgeführte Prüfungen

- Python-Syntaxprüfung mit `python -m py_compile .\server.py`
- JavaScript-Syntaxprüfung mit `node --check`
- JSON-Validierung von `study-plan.json`
- HTTP-Smoke-Test des lokalen Servers
- Prüfung ungültiger Mensa-IDs mit erwarteter HTTP-400-Antwort
- VS-Code-Diagnose ohne gemeldete Fehler oder Warnungen
- Manuelle Prüfung von Datumswechsel, Campuswechsel, Wetterfehlern und Standortfreigabe

Die externen Dienste bleiben abhängig von Netzwerk, Verfügbarkeit und ihrer aktuellen Antwortstruktur. Das Testkonzept ist in `testkonzept.md` dokumentiert.

Qualitäts- und Umsetzungsstand

- CampusKompass wird mit `python .\campuskompass\server.py` gestartet.
- Die externen Karten-, Wetter- und Mensadienste benötigen eine Netzwerkverbindung.

Start der Anwendungen

CampusKompass

```
cd ".\campuskompass"
python .\server.py
```

Danach: http://127.0.0.1:8000/

Fazit

- CampusKompass bündelt studienbezogene Informationen in einer Weboberfläche.
- Die MoSCoW-Priorisierung trennt MVP-Funktionen von Erweiterungen.
- Nächste Ausbaustufen sind zusätzliche Standorte, Datenquellen und automatisierte Browser-Tests.